

Gauge Pressure Classification Using TinyML

Complete Project Development and Deployment Explanation

Project Objective

The project was developed to classify the pressure shown on an analog pressure gauge using machine learning and eventually deploy the trained model on an Arduino Nano 33 BLE Sense. The project follows a complete machine-learning pipeline in which labeled gauge images are prepared, a CNN model is trained on a computer, the model is optimized for TinyML, converted to TensorFlow Lite, and then deployed to the Arduino.

Creating the Dataset and JSON File

The first step was to organize the gauge images and create labels for them. A JSON file named labels.json was created inside the dataset folder. The purpose of this file was to connect every image with its corresponding information. Each record contains details such as the image path, gauge type, minimum pressure, maximum pressure, needle angle, pressure range and status.

The dataset contained 290 labeled image records. The Python training program reads the JSON file and obtains the image path and pressure label from each record.

```
{"image": "0-100/1.png", "gauge_type": "0-100", "min_pressure": 0, "max_pressure": 100, "needle_angle": 150, "pressure": "0-20", "status": "Unsafe (Low Pressure)"}
```

Loading and Preparing the Images

After reading labels.json, the Python program loads each image using OpenCV. The images may have different dimensions, so every image is resized to 96×96 pixels. The pixel values are converted to floating-point values and divided by 255.0 so that the image values are normalized between 0 and 1.

The pressure labels are text values, so LabelEncoder is used to convert the labels into numerical class values that can be used by the neural network.

Splitting the Dataset

The dataset was divided into training and testing data. An 80%/20% split was used. From the 290 images, 232 images were used for training and 58 images were used for testing.

Training the First CNN Model

The first model was a Convolutional Neural Network. It was designed to learn visual features from the gauge images and classify them into the pressure classes.

```
96×96×3 → Conv2D(32) → MaxPooling → Conv2D(64) → MaxPooling → Flatten → Dense(128)
→ Dense(11)
```

The model was compiled using the Adam optimizer and sparse categorical cross-entropy loss. It was trained for 20 epochs with a batch size of 16.

The first model contained approximately 3.99 million parameters. The trained Keras model was approximately 47 MB in size.

Evaluating the First Model

After training, the model was tested using the 58 test images. It correctly classified 35 images and incorrectly classified 23 images. The resulting test accuracy was approximately 60.34%.

The model was then saved as model.keras.

Why the First Model Was Not Used for Arduino Deployment

The first model was too large for practical deployment on the Arduino. When it was converted to TensorFlow Lite, the model was still approximately 15 MB. The large model size was mainly caused by the Flatten layer followed by the Dense layer with 128 neurons.

Because the target is a microcontroller, a much smaller model was required. Therefore, instead of trying to deploy the large model directly, a smaller CNN was created.

Creating the Smaller CNN

A lightweight CNN was developed to reduce the memory and storage requirements. The number of convolution filters was reduced and the large Flatten-to-Dense section was replaced with GlobalAveragePooling followed by a smaller Dense layer.

```
96×96×3 → Conv2D(16) → MaxPooling → Conv2D(32) → MaxPooling → Conv2D(32) →  
GlobalAveragePooling → Dense(32) → Dense(11)
```

This smaller model was trained using the same general dataset and classification objective.

Converting the Small Model to INT8 TensorFlow Lite

After training the smaller model, it was converted into TensorFlow Lite format. TensorFlow Lite optimization was enabled and a representative set of training images was used for quantization. The model was configured to use INT8 input and INT8 output.

The resulting file was named model_quantized.tflite and its size was 23,984 bytes, approximately 23.42 KB. This was a very large reduction compared with the original approximately 15 MB TFLite model.

Creating model.h

The Arduino program cannot directly use the TensorFlow Lite file in the same way as the Python environment. Therefore, the quantized TFLite model was converted into a C/C++ header file called model.h.

The model bytes were stored in an unsigned character array together with the model length. The resulting model.h file appeared as approximately 153 KB in Windows Explorer because the binary model bytes were represented as hexadecimal text.

Deploying the Model to Arduino

An Arduino sketch named Gauge_Deployment was created. The generated model.h file was placed in the same sketch folder as the Arduino .ino file.

The Arduino program uses TensorFlow Lite Micro. It includes model.h, loads the TensorFlow Lite model, checks the model schema, registers the operations required by the neural network, creates the MicroInterpreter and then calls AllocateTensors() to prepare the memory required for inference.

```
labels.json → training → model_small.keras → model_quantized.tflite → model.h →  
Arduino
```

First Arduino Error: Missing MEAN Operation

During the first deployment attempt, the Arduino successfully loaded the model but reported that the MEAN operation was missing. This happened because the smaller CNN used GlobalAveragePooling, which was represented as a MEAN operation in the TensorFlow Lite model.

The Arduino operator resolver initially contained the other required operations but did not contain MEAN. The resolver was therefore updated by adding AddMean() and increasing the resolver capacity.

After this change, the previous MEAN operator error was removed. This confirmed that the model and the required operator could be recognized by TensorFlow Lite Micro.

Current Arduino Problem

After fixing the MEAN operation, the Arduino Serial Monitor shows:

```
Loading model...  
Model loaded successfully.  
Allocating tensors...
```

The program does not proceed beyond Allocating tensors. It therefore does not reach the actual inference stage.

Why It Is Not Working

The current problem is occurring during TensorFlow Lite Micro tensor allocation. Loading a model and running a model are separate stages. The fact that the model loads successfully means that the model.h file is being recognized by the TensorFlow Lite Micro runtime.

The 23.42 KB size refers only to the stored quantized model. During inference, TensorFlow Lite Micro also needs RAM for intermediate tensors, activation buffers, the tensor arena, interpreter data, the Arduino program, stack and other runtime requirements.

Therefore, even though the model file was successfully reduced from approximately 15 MB to 23.42 KB, the complete runtime memory requirement can still be much larger. The current network and runtime configuration are not completing tensor allocation on the Arduino.

Increasing the tensor arena size was attempted, but the program still stopped at the allocation stage. Because tensor allocation does not complete, the program cannot reach inference.

Why the Camera Was Not Tested

The camera was deliberately not integrated during the final debugging stage. The first goal was to verify that the TensorFlow Lite model could run on the Arduino independently of the camera. Since the failure occurs before inference, connecting the camera would not solve the current problem.

Final Project Status

The project successfully completed the dataset preparation, JSON labeling, image preprocessing, CNN training, model evaluation, lightweight model development, INT8 quantization, TensorFlow Lite conversion, model.h generation and embedded model loading.

The remaining incomplete part is on-device inference. The Arduino successfully loads the model but cannot complete tensor allocation for the current network and runtime configuration.

Conclusion

The project reached the embedded deployment stage but could not be completed as a real-time Arduino inference system within the available development time. The main limitation was not the creation or conversion of the machine-learning model; it was the runtime memory/resource requirement during TensorFlow Lite Micro tensor allocation.

For a future version, the model can be optimized further by reducing the input resolution, reducing the number of layers and filters, simplifying the network operations and measuring the exact tensor arena requirement. After successful tensor allocation, inference can be tested with a fixed image and then the camera can be integrated.

Source note: The training workflow described above follows the supplied training program, which reads dataset/labels.json, loads and resizes images, normalizes them, encodes labels, splits the data, trains the CNN and saves model.keras.